

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering)

ভারতীয় প্রৌদ্যোগিকী সংস্থান, মাদ্রাজ
(Indian Institute of Technology, Madras)

সপ্তাহ – 01 বক্তৃতা- 11

অপারেটিং সিস্টেম (প্রসেসস)

নমস্কার এবং এই ভিডিওটিতে স্বাগত জানাই। তাই এই ভিডিওতে আমরা প্রসেসগুলি দেখব, যা সম্ভবত অপারেটিং সিস্টেমগুলির সবচেয়ে গুরুত্বপূর্ণ অংশ। সুতরাং, প্রসেসগুলি আমরা জানি এক্সিকিউশনরত একটি প্রোগ্রাম। তাই আমরা আজ দেখবো কিভাবে অপারেটিং সিস্টেম প্রসেস পরিচালনা করে।

(স্লাইডটাইপ পড়ুন: 00:38)

তাই আসুন শুরু করা যাক এই স্ক্রিনে "হ্যালো ওয়ার্ল্ড" প্রিন্টিংয়ের একটি বিখ্যাত উদাহরণ দিয়ে। সুতরাং, যখন gcc দ্বারা কম্পাইলে হ্যালো ডট সি একটি এক্সিকিউটেবল এ ডট আউট তৈরি হয়। সুতরাং যখন একটি ডট আউট এক্সিকিউট হয় একটি প্রসেস তৈরি হয়। এই প্রসেসটির অংশ RAM-র মধ্যে থাকবে এবং এটি একটি ভার্সুয়াল ঠিকানা ম্যাপ দ্বারা সনাক্ত করা হবে। তাই ভার্সুয়াল ঠিকানার ম্যাপটি পরপর ঠিকানার মেমরি অবস্থান যা 0 থেকে শুরু করে MAX_SIZE পর্যন্ত হয়। সুতরাং, এই ভার্সুয়াল ঠিকানা ম্যাপের মধ্যে প্রসেসের বিভিন্ন দিক যেমন ইন্সট্রাকসন, গ্লোবাল এবং স্ট্যাটিক ডেটা, হীপ, সেইসাথে স্ট্যাক আছে।

(স্লাইডটাইপ পড়ুন: 01:28)

যেমন আমরা আগের ভিডিও দেখেছি, প্রসেসের ভার্সুয়াল অ্যাড্রেস স্পেস বা ভার্সুয়াল অ্যাড্রেস ম্যাপটি সমান আকারের ব্লকগুলিতে বিভক্ত। তাই সাধারণত প্রতিটি ব্লকের মাপ 4 কিলো বাইট হয়। আবার প্রতিটি প্রসেসের একটি মেমরি একটি প্রসেস পেজ টেবিল আছে যা একটি সংশ্লিষ্ট পেজ ফ্রেমের মধ্যে প্রতিটি ব্লক প্রসেস হবে। আমরা দেখেছি যে, র‍্যাম আকারে পেজ ফ্রেমের মধ্যে বিভক্ত যা 4 KB ব্লক আকারের মত। এবং এই পেজ ফ্রেমগুলি বাস্তব কোড এবং প্রসেসের যা ডেটা এক্সিকিউট হচ্ছে সেগুলো ধারণ করে। আমরা এইগুলি পূর্ববর্তী ভিডিওতে

দেখেছি। কিন্তু আমাদের জিজ্ঞেস করতে হবে যে প্রশ্নটি তা হল; অপারেটিং সিস্টেম বা কার্নেলটি এই সমগ্র স্কিমের কোথায় থাকে। তাই আমরা জানি কার্নেলটি অন্য একটি সফটওয়্যার এবং এটি এক্সিকিউটের জন্য RAM এ উপস্থিত থাকতে হবে। সুতরাং, বেশিরভাগ অপারেটিং সিস্টেম যেমন লিনাক্সের পাশাপাশি অপারেটিং সিস্টেমে যা আমরা অধ্যয়ন করছি তা হল xv6, কার্নেল 1, 2, 3 পেজ ফ্রেম থেকে শুরু করে মেমরির নীচের অংশে থাকে। শুধু প্রতিটি অন্যান্য পেজ ফ্রেমের মত দ্বিতীয় কার্নেলটি সমান আকারের পেজ ফ্রেমে বিভক্ত। এখন আমরা সিস্টেমের ভার্চুয়াল অ্যাড্রেসিং ব্যবহার করছি, কার্নেলের সাথে সংশ্লিষ্ট পেজ ফ্রেম প্রসেসটির ভার্চুয়াল অ্যাড্রেস স্পেসে ম্যাপ করা হয়। সুতরাং, কার্নেল কোড এবং ডেটা MAX_SIZE এর উপরে এবং নীচের সীমার নীচে বিদ্যমান। এখন, প্রসেস পেজ টেবিলের আবার এই ম্যাপের সাথে সম্পর্কিত এন্ট্রি রয়েছে। উদাহরণস্বরূপ 7, 8, 9 এবং 10 ব্লকগুলির সাথে সংশ্লিষ্ট যাদের কার্নেল কোড এবং ডেটা রয়েছে এবং পেজ টেবিলের সাথে আমাদেরকে বলে যে তারা 1, 2, 3 এবং 4 নম্বার পেজ ফ্রেমে ম্যাপ করা হয়।

(স্লাইডটাইপ পড়ুন: 03: 42)

এখন, আমরা এই নির্দিষ্ট ভার্চুয়াল অ্যাড্রেস স্পেসকে দুটি ভাগে বিভক্ত করতে পারি। একটি ইউজার স্পেস যা নীল এলাকার সাথে সম্পর্কিত আর যা ইউজার প্রসেসকে ধারণ করে, কোড, ডেটা এবং স্ট্যাক এবং হীপের মত অন্যান্য সেগমেন্টগুলির মধ্যে রয়েছে। আবার, কার্নেলের স্থান আছে যা কার্নেল কোড ডেটা এবং কার্নেলের অন্যান্য দিকগুলির সাথে সম্পর্কিত। সুতরাং, MAX_SIZE ব্যবহারকারীর স্থান এবং কার্নেল স্থানের মধ্যে সীমানা নির্ধারণ করে। একটি ইউজার প্রোগ্রাম এই ইউজার স্পেসে যেকোনো কোড বা ডাটা অ্যাক্সেস করতে পারে। ব্যবহারকারী প্রোগ্রামটি কার্নেলের স্থানে কিছু অ্যাক্সেস করতে পারে না। অন্যদিকে, কার্নেলটি কোড এবং পাশাপাশি কার্নেলের স্থান এবং ব্যবহারকারীর স্থান উভয়ের ডাটাও অ্যাক্সেস করতে পারে। সুতরাং, এটি ইউজার স্পেস প্রোগ্রামগুলিকে ক্ষতিকারক ডাটা পরিবর্তন বা কার্নেল স্ট্রাকচারগুলি পরিবর্তন করা থেকে বাধা দেয়।

(স্লাইডটাইপ পড়ুন: 04:51)

তাই, আরেকটি বিষয় লক্ষ্য করা যায় যে কার্নেলের অ্যাড্রেসের মধ্যে একটি পরপর ম্যাপিং রয়েছে প্রসেসের ভার্চুয়াল স্পেসের থেকে সংশ্লিষ্ট ফিজিক্যাল ফ্রেম পর্যন্ত যেখানে কার্নেলে ম্যাপ করা হয়। উদাহরণস্বরূপ, কার্নেল 7, 8, 9 এবং 10 ব্লকগুলি সংহত পেজগুলির ফ্রেমে 1, 2, 3, এবং 4-এ ম্যাপ করা হয়। সুতরাং, কেন এই সংহত ম্যাপিং আসলে ব্যবহৃত হয়? এর সর্বাপেক্ষা গুরুত্বপূর্ণ দিকটি হল এই সংহত ম্যাপিংটি দেওয়া যাতে কার্নেলের ভার্চুয়াল ঠিকানা থেকে ফিজিক্যাল ঠিকানা এবং তদ্বিপরীত রূপান্তর করা সহজ হয়। উদাহরণস্বরূপ, কার্নেলের পেজ ফ্রেমের মধ্যে সংশ্লিষ্ট ফিজিক্যাল ঠিকানাতে ভার্চুয়াল অ্যাড্রেস থেকে রূপান্তর করার জন্য MAX_SIZE দ্বারা একটি সাধারণ বিয়োগই কৌশল হবে। উদাহরণস্বরূপ, xv6 তে যেখানে MAX_SIZE 0x80000000 হিসাবে সংজ্ঞায়িত করা হয়, 0x80124345 এর একটি ভার্চুয়াল অ্যাড্রেস MAX_SIZE থেকে বিয়োগ করে

সংশ্লিষ্ট ফিজিক্যাল ঠিকানায় রূপান্তরিত হতে পারে। সুতরাং, ফিজিক্যাল ঠিকানাটি কেবল 0x00124345 হিসাবে লিখিত হবে। অনুরূপভাবে, কার্নেলের ফিজিক্যাল অ্যাড্রেস সংশ্লিষ্ট কার্নেল কোড এবং সংশ্লিষ্ট কার্নেল পেজ ফ্রেমের একটি তথ্য ভার্সুয়াল ঠিকানাতে রূপান্তর করা যাবে যাতে MAX_SIZE যোগ করতে হবে। উদাহরণস্বরূপ, এই ক্ষেত্রে ফিজিক্যাল ঠিকানা 0x00124345টি এই ভার্সুয়াল ঠিকানাতে রূপান্তর করা যেতে পারে MAX_SIZE-টিকে যোগ করে পাওয়া যায় - 0x80124345 ।

(স্লাইডটাইপ পড়ুন: 06:52)

সুতরাং, আমাদের সিস্টেমে একাধিক প্রসেস থাকলে কি হবে? প্রতিটি প্রসেসের ভার্সুয়াল অ্যাড্রেস স্পেসে কার্নেল স্পেসটি ম্যাপ করা হয়। উদাহরণস্বরূপ, MAX_SIZE এর উপরে এবং ম্যাক্স লিমিটের নিচের কার্নেল স্পেস সমস্ত প্রসেসগুলিতে উপস্থিত থাকে। অনুরূপভাবে, প্রতিটি প্রসেসের পেজ টেবিলটি কার্নেল পেজ টেবিল এবং সংশ্লিষ্ট পেজ ফ্রেমগুলি যা কার্নেলটি দখল করে থাকে তার মধ্যে একটি অভিন্ন ম্যাপিং বর্তমান, যেমনটি এই কয়েকটি এন্ট্রিগুলির পাশাপাশি এই কয়েকটি এন্ট্রিগুলিতেও দেখা যায়। এখন লক্ষ্য করা যেতে পারে যে, প্রতিটি প্রসেসের ভার্সুয়াল অ্যাড্রেস স্পেসে কার্নেলের বিভিন্ন এন্ট্রি রয়েছে। যাইহোক, সব প্রসেসগুলিই তাদের র‍্যামের একই পেজ ফ্রেমে কার্নেল স্থান মাপবে। সুতরাং, এর অর্থ হল, আমাদের RAM- এ উপস্থিত কার্নেলের একটিমাত্র কপি আছে। যাইহোক, কার্নেল কোড এবং ডেটা সংক্রান্ত প্রতিটি প্রসেসের পেজ টেবিলে একাধিক অভিন্ন এন্ট্রি হতে পারে।

(স্লাইডটাইপ পড়ুন: 08:1২)

এখন আমরা দেখেছি যে কার্নেলটি RAM- র মধ্যে বিদ্যমান এবং সেখানে প্রতিটি প্রসেসে ভার্সুয়াল অ্যাড্রেস স্পেসের ম্যাপ পাওয়া যায়, এখন আমরা দেখব যে কার্নেলটির সাথে সম্পর্কিত কোন মেটাডেটা আছে সিস্টেমের মধ্যে সংশ্লিষ্ট প্রতিটি প্রসেসে থাকে। সুতরাং, সিস্টেমে প্রতিটি প্রসেসে 3 টি মেটাডেটা থাকে; প্রসেস নিয়ন্ত্রণ ব্লক, ব্যবহারকারীর প্রসেসের জন্য একটি কার্নেল স্ট্যাক এবং ব্যবহারকারীর প্রসেসের জন্য সংশ্লিষ্ট পেজ টেবিল। সুতরাং, সিস্টেমে চালানো প্রতিটি প্রসেস এই তিনটি ব্লক থাকবে যা সেই প্রসেসের জন্য অনন্য। তাই আমরা ইতিমধ্যে একটি পেজ টেবিল ম্যাপে দেখিয়েছি যে ব্যবহারকারীর ভার্সুয়াল অ্যাড্রেসযোগ্য স্থানটি সংশ্লিষ্ট পেজ ফ্রেমগুলির যে প্রসেসটি দখল করে আছে তার সাথে তুলনা করে। এখন আমরা দেখব অন্য দুটি মেটাডেটা।

(স্লাইডটাইপ দেখুন: 09:06)

তাই, আমরা শিখেছি যে প্রতিটি প্রসেসের সাথে সংশ্লিষ্ট বিভিন্ন সেগমেন্ট রয়েছে এবং এক গুরুত্বপূর্ণ সেগমেন্ট হল প্রসেসটির স্ট্যাক। তাই ব্যবহারকারীর স্থানটিতে উপস্থিত এই প্রসেস স্ট্যাকের মধ্যে স্থানীয় ভেরিয়েবল এবং ফাংশন কলগুলির তথ্যও থাকবে। সুতরাং একে আমরা এখন বলব ইউজার স্পেস স্ট্যাক। ইউজার স্পেস স্ট্যাক ছাড়াও প্রত্যেকটি প্রসেসের থাকে কার্নেল স্পেস স্ট্যাক যা কার্নেল স্ট্যাক নামেও পরিচিত হবে। তাই এই কার্নেল স্ট্যাক ব্যবহার করা হয় যখন কার্নেল একটি প্রসেসের প্রেক্ষিতে এক্সিকিউট হয়। উদাহরণস্বরূপ, যখন প্রসেসটি একটি সিস্টেম কল এক্সিকিউট করে তখন এটি কিছু কার্নেল কোড সম্পাদন করে এবং এই কার্নেল কোডটি কার্নেল স্ট্যাক ব্যবহার করবে কারণ এটি স্থানীয় ভেরিয়েবল এবং ফাংশন কলগুলি জন্য। এছাড়াও এই কার্নেল স্ট্যাক অন্যান্য প্রজেক্টের জন্য ব্যবহার করা হয় যেমন প্রসেসের প্রেক্ষাপট এবং সঞ্চয় করার জন্য। স্থানীয় এবং অটো ভেরিয়েবলের জন্য এবং ফাংশন কলগুলির জন্য স্ট্যাকের স্ট্যান্ডার্ড ব্যবহারের পাশাপাশি কার্নেল স্ট্যাকগুলি প্রসেসটির প্রেক্ষাপটে সংরক্ষণের একটি গুরুত্বপূর্ণ ভূমিকা পালন করে যা প্রসেসটি সময়ের পরে পুনরায় চালু করতে দেয়। সুতরাং, কেন আমাদের দুটি পৃথক স্ট্যাক আছে? কেন আমাদের প্রসেসের জন্য ইউজার স্ট্যাক এবং কার্নেল স্ট্যাক আছে? আমরা যে সুবিধাটি অর্জন করি তা হল যে কার্নেলটি কাজ করতে পারে এমনকি যদি ব্যবহারকারী স্ট্যাকটি নষ্ট হয়ে যায়। অতএব স্ট্যাককে লক্ষ্য করে এমন আক্রমণগুলি, যেমন বাফার ওভারফ্লো আক্রমণ যেমন একটি ক্ষেত্রে কার্নেলকে প্রভাবিত করবে না। সুতরাং, আসুন আমরা PCB এর কিছু গুরুত্বপূর্ণ উপাদান দেখি। সুতরাং এই বিশেষ গঠন xv6 অপারেটিং সিস্টেম PCB থেকে নেওয়া হয় যা struct proc হিসাবে সংজ্ঞায়িত করা হয়। এই বিশেষ কাঠামোর কিছু গুরুত্বপূর্ণ উপাদান বা দিক হল 'sz' যা প্রসেস মেমোরির আকার, 'pgdir' যা প্রসেসের জন্য পেজ ডিরেক্টরির একটি পয়েন্টার। 'kstack' যা কার্নেল স্ট্যাকের একটি পয়েন্টার যা আমরা আগে কয়েকটি স্লাইডে সংজ্ঞায়িত করেছি। এবং অন্যান্য দিক যেমন প্রসেস দ্বারা খোলা ফাইলগুলির একটি তালিকা, প্রসেসের বর্তমান কার্যকরী ডিরেক্টরি এবং এক্সিকিউটেবল নাম; উদাহরণস্বরূপ, আমাদের উদাহরণে একটি ডট আউট। সুতরাং, আমরা পরবর্তী স্লাইডে এই অন্যান্য বিষয়গুলিকে দেখবো।

(স্লাইডটাইপ পড়ুন: 12:04)

তাই প্রতিটি প্রসেসের সাথে সংশ্লিষ্ট PCB-তে একটি গুরুত্বপূর্ণ এন্ট্রি হলো PID বা প্রসেস আইডেন্টিফাইয়ার। সুতরাং এই প্রসেসের জন্য একটি শনাক্তকারী মূলত একটি পূর্ণসংখ্যা হিসাবে সংজ্ঞায়িত এবং প্রতিটি প্রসেসের একটি অনন্য PID হবে। সুতরাং সাধারণত, সংখ্যা ক্রমান্বয়ে এমনভাবে বৃদ্ধি করা হবে যে প্রসেসটি তৈরি হলে এটি একটি অনন্য সংখ্যা পায়।

(স্লাইডটাইপ পড়ুন: 12:35)

পিসিবি আরেকটি গুরুত্বপূর্ণ দিক হল প্রসেসের অবস্থা। তাই প্রসেস তৈরির সময় থেকে শুরু করে যে সময় পর্যন্ত এটি বর্তমান তার বিভিন্ন অবস্থা রয়েছে। যেমন নিউ (NEW), রেডি (READY), ব্লক স্টেট (BLOCK state) বা

রানিং স্টেট (Running state)। Xv6 বিভিন্ন নামে এগুলোকে ডাকে যেমন, নিউকে বলা হয় ক্রণ বলা হয় মানে এটি একটি নতুন প্রসেস বর্তমানে তৈরি করা হচ্ছে, আবার রেডিকে বলা হয় রানবল(Runnable) যা চালানোর জন্য প্রস্তুত, ঘুমন্ত বলা হয় ব্লক অবস্থাকে এবং মূলত একটি I / O এর জন্য অবরুদ্ধ। সুতরাং, কিভাবে এবং কখন একটি প্রসেস আসলে একটি অবস্থা থেকে অন্যটিতে যায়? সুতরাং একটি নতুন প্রসেস যখন তৈরি হয় তখন এটি প্রাথমিকভাবে NEW নামে পরিচিত স্টেট হয়। যখন এটি রান করার জন্য প্রস্তুত হয় তখন স্টেটটি READY স্টেট হিসাবে পরিচিত হয়, এবং যখন এটি প্রসেসরে চলতে থাকে তখন এটি রানিং স্টেটে স্থানান্তরিত হয়। কিছু সময় ধরে চালানোর পর প্রসেসটিকে প্রসেসর থেকে থামানো হয় আর অন্য প্রসেসকে চালানোর অনুমতি দেওয়া হয়, এবং এই ক্ষেত্রে এটি রানিং স্টেট থেকে রেডি স্টেটে ফিরে যায়। এখন, প্রসেস সঞ্চালনের সময় ধরি কিছু I / O অপারেশনের প্রয়োজন আছে। উদাহরণস্বরূপ প্রসেস একটি স্ক্যানার আহ্বান করতে পারে যা ব্যবহারকারীকে কীবোর্ডের মাধ্যমে কিছু প্রবেশ করতে বলে। এই ক্ষেত্রে একটি প্রসেস একটি রানিং স্টেট থেকে একটি ব্লক স্টেটে সরানো হবে। সুতরাং, ঘটনাটি সংঘটিত না হওয়া পর্যন্ত প্রসেসটি ব্লক স্টেটে থাকবে। উদাহরণস্বরূপ, যখন ব্যবহারকারী কীবোর্ডের মাধ্যমে কিছু প্রবেশ করান, তখন প্রসেসটিকে ব্লক স্টেট থেকে পুনরায় রেডি স্টেটে সরানো হয়। এবং এই প্রসেসকে একটি স্টেট থেকে আরেকটি স্টেটে সরানোর প্রক্রিয়া, রেডি থেকে রানিং, রানিং থেকে বা রেডি অথবা রানিং থেকে ব্লক এবং তারপর রেডি এটা প্রসেসটির সমগ্র জীবন চক্র মাধ্যমে চলতে থাকে। শেষে যখন প্রসেসটি বিদ্যমান থাকে বা শেষ হয়ে যায় তখন এটি একটি এক্সিট(EXIT) স্টেট হিসাবে পরিচিত হয়, এটি এই ডায়াগ্রামে দেখানো হয়নি। সুতরাং, আপনি আসলে xv6 কোড proc dot h সন্ধান করতে পারেন এবং যা আপনাকে বিভিন্ন স্টেটের সম্পর্কে আরও তথ্য দিতে বলবে। সুতরাং, এই রেডি স্টেট কি?

(স্লাইডটাইপ দেখুন: 15:19)

অপারেটিং সিস্টেমে প্রসেসের একটি সারি বজায় রাখা হয় যেগুলি রেডি স্টেটে রয়েছে। যখন কোনও ইভেন্ট যেমন টাইমার ইন্টারাপ্ট ঘটে, তখন অপারেটিং সিস্টেমের মধ্যে একটি মডিউল যেটি সিপিইউ সিডিউলার হিসাবে পরিচিত তাকে চালানো হয়। এই সিপিইউ সিডিউলার তারপর এই প্রস্তুত প্রসেসের সারি স্ক্যান করে এবং তারপর একটি প্রসেসকে নির্বাচন করে যেটি তারপর প্রসেসরে এক্সিকিউট হয়। এই নির্বাচিত প্রসেস তারপর এটি রেডি স্টেট থেকে রানিং স্টেটে আসে। রানিং প্রসেসটি পরবর্তী টাইমার ইন্টারাপ্ট না হওয়া পর্যন্ত এক্সিকিউট হতে থাকবে এবং সমগ্র চক্রটি নিজেই পুনরাবৃত্তি হবে।

(স্লাইডটাইপ পড়ুন: 16:03)

পিসিবি আরেকটি এন্ট্রি হল পয়েন্টার যা ট্র্যাপফ্রেম এবং কন্টেক্সট হিসাবে পরিচিত। সুতরাং, এই ট্র্যাপফ্রেম (trapframe) এবং পাশাপাশি কন্টেক্সট কার্নেল স্ট্যাকের অংশ এবং এই চিত্রে যেরকম দেখা যাচ্ছে তাদের চলমান প্রসেসের বর্তমান অবস্থা সম্পর্কে অনেক তথ্য আছে। উদাহরণস্বরূপ, এটি স্ট্যাক মেট সেগমেন্ট, স্ট্যাক পয়েন্টার,

ফ্লাগ রেজিস্টার, কোড সেগমেন্ট, ইন্সট্রাকসন পয়েন্টার ইত্যাদি বলে। সুতরাং, এই বিশেষ ট্র্যাপফ্রেম এবং কন্টেক্সট ব্যবহার করা হয় যখন একটি কন্টেক্সট সুইচ এর পরে একটি প্রসেস পুনরায় আরম্ভ করা হয়।

(স্লাইডটাইম পড়ুন: 16:43)

অতএব, কিভাবে এই পিসিবি xv6 তে সংরক্ষিত হয়? সুতরাং, xv6 মধ্যে ptable হিসাবে পরিচিত একটি স্ট্রাকচার সংজ্ঞায়িত করা হয়। তাই এই স্ট্রাকচার struct procs একটি অ্যারের মধ্যে আছে, তাই struct procs আসলে xv6 মধ্যে পিসিবি স্ট্রাকচার। সুতরাং অ্যারের আছে NPROC এন্ট্রি, যেখানে NPROC 64 হিসাবে সংজ্ঞায়িত করা হয়। সুতরাং, xv6 এ তৈরি করা প্রতিটি প্রসেসের এই বিশেষ অ্যারেতে একটি এন্ট্রি থাকবে। সুতরাং, আপনি এই বিশেষ স্ট্রাকচার সম্পর্কে আরো তথ্য থাকতে পারে xv6 কোড proc ডট c তে এবং স্ট্রাকচার ptable এ। এ ছাড়াও params ডট এইচ হল xv6 এর একটি ফাইল যা NPROC কী তা নির্ধারণ করে। সুতরাং, এটি আমাদের অপারেটিং সিস্টেমের মধ্যে কিভাবে প্রসেসগুলি পরিচালিত হয় তার সংক্ষিপ্ত পরিচয় দেয়। পরবর্তী ভিডিওতে, আমরা দেখব কিভাবে প্রসেসটি তৈরি করা হয়, এক্সিকিউট করা হয় এবং সিস্টেম থেকে প্রস্থান করে। ধন্যবাদ।